



Data Handling: Import, Cleaning and Visualisation

Lecture 4: From Text to Data. Understanding R Data Structures

Dr. Aurélien Sallin

2024-10-09

Recap

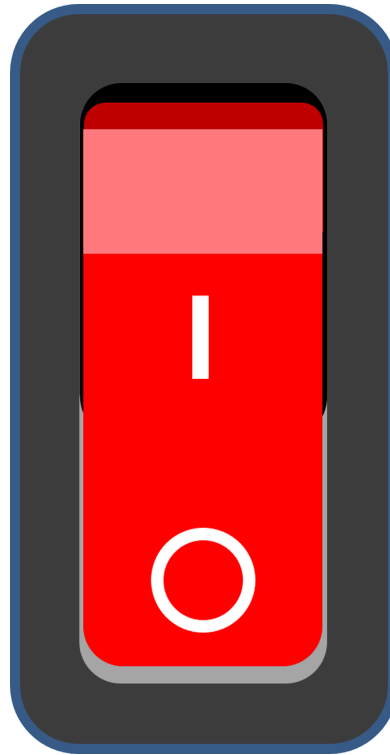
Goals of last lecture

1. Get the basics of **data processing**
2. Learn what **binary** and **hexadecimal** systems are
3. See why **encoding** is crucial for any data project
4. Realize that both code and data are just **text** at their core
5. Execute your first website scraper in R.

The binary system

Microprocessors can only represent two signs (states):

- 'Off' = 0
- 'On' = 1



The binary and the hexadecimal counting frames

Binary:

- Only two signs: 0, 1.
- Base 2.
- Columns: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, and so forth.

Hexadecimal:

- 16 symbols
- Base 16.
- 0-9 (used like in the decimal system) and A-F (for the numbers 10 to 15).

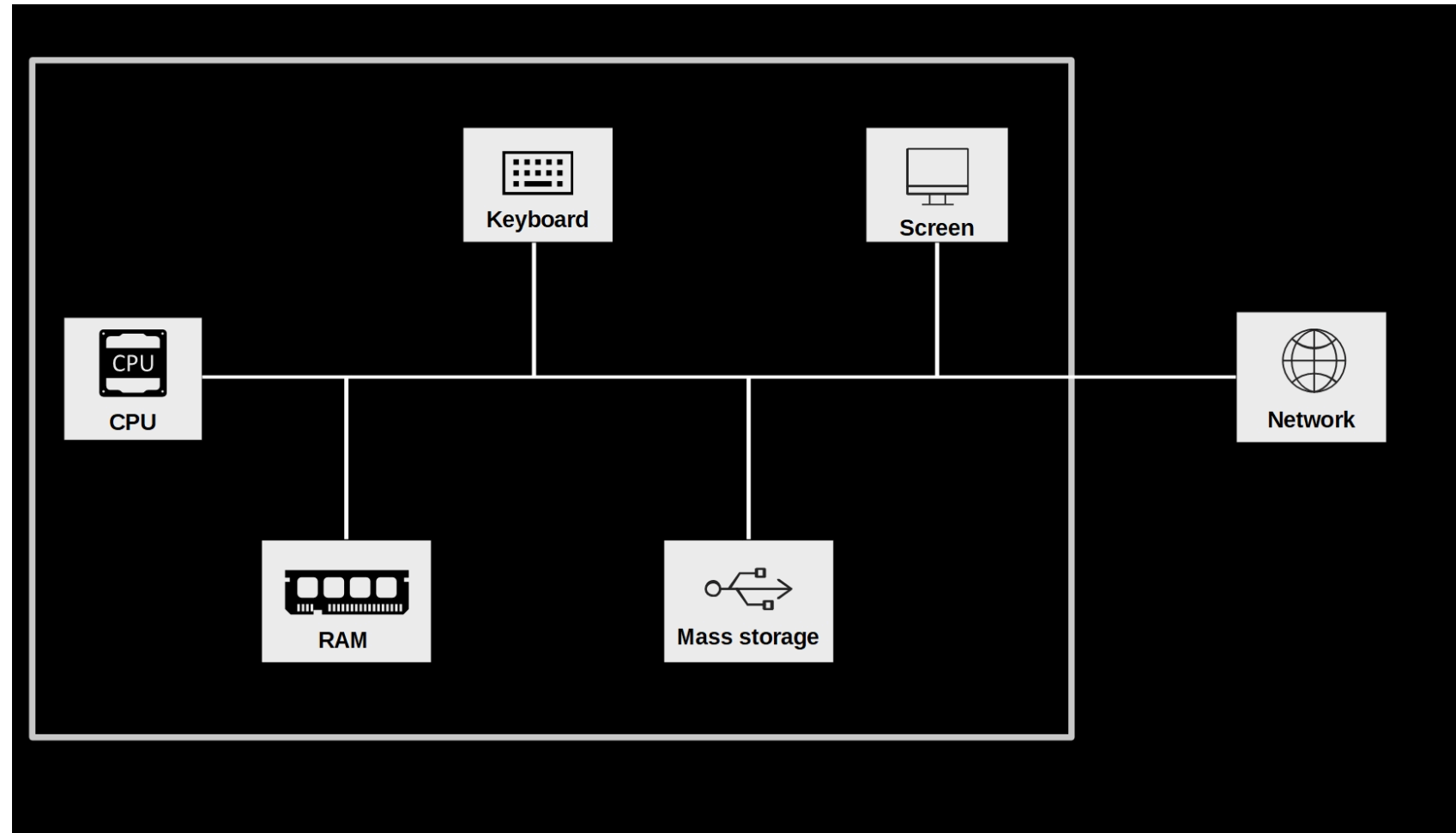
Computers and text

How can a computer understand text if it only understands 0s and 1s?

- *Standards* define how 0s and 1s correspond to specific letters/characters of different human languages.
- These standards are usually called *character encodings*.
- Coded character sets that map unique numbers (in the end in binary coded values) to each character in the set.
- For example, ASCII (American Standard Code for Information Interchange) or *utf-8*.

The image shows the word "ASCII" in a large, bold, blue, monospace-style font. The letters are slightly pixelated, giving it a digital or retro appearance.

Components of a standard computing environment



Exercise: a website scraper

Last time, we saw that a webpage is nothing but **text** (HTML). We looked at its hexadecimal representation. The size in RAM of our “economist” html object was **468048 bytes**.

From hexadecimal

```
library(httr)
economist <- GET("https://www.economist.com/")
head(content(economist, as = "raw"), 20)
```

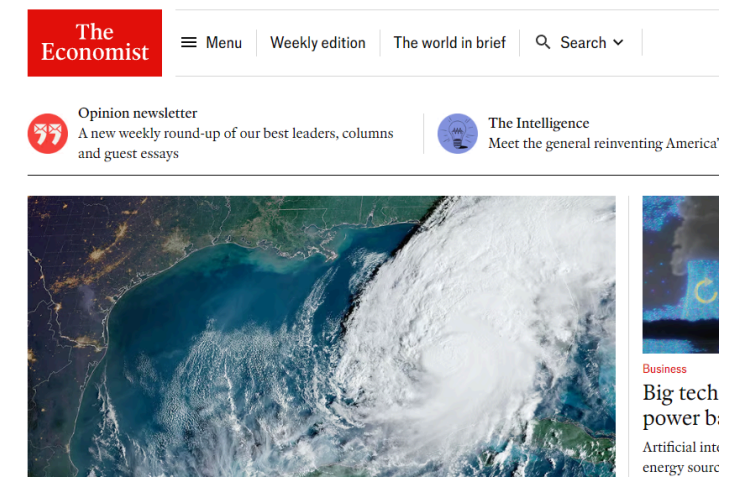
```
[1] 3c 21 44 4f 43 54 59 50 45 20 68 74 6d 6c
3e 3c 68 74 6d 6c
```

... to html...

```
head(content(economist, as = "text"))
```

```
"<!DOCTYPE html><html
lang=\"en\"><head>
```

... to website



Warm-up

Warm-up 1&2

What is 234 in binary? Please answer exact response with format 00000000.

What is 22C (hex) in decimal?

Warm-up 3

Consider the following R function:

```
muLtip <- function(n, limit) {  
  
  product <- 1  
  counter <- 1  
  
  while (counter <= n) {  
    product <- product * counter  
    if (product > limit) {  
      return(product)  
    }  
    counter <- counter + 1  
  }  
  return(product)  
}
```

What is `muLtip(5,20)`?

Warm-up 4

Translating between binary and hexadecimal is straightforward: each hexadecimal digit maps directly to a group of **4 binary digits**

Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
...	
D	1101
E	1110
F	1111

Today

Goals of today's lecture: from text to data

Our story:

- Computers store data as bits (Bits → Bytes → Encoded Text)
- Text encodes meaning
- Files encode structure (→ Structured Files (CSV))
- R parses structure into memory. (→ R Parsing → R Data Structures)

Goals:

- Understand how we **import** and structure data from text files
- Learn **data structures** in R
- Understand **rectangular data formats**

From text to data structure

Structured Data Formats

- Now that we understand how text and encodings work, we can see how text can represent structured data.
- We are used to not thinking about the formats of our data... and let the computer choose what to do with data when we click on a file.

All data are text files, but with standardized structure.

- Special characters define the structure.
- More complex **syntax**, more complex structures can be represented...

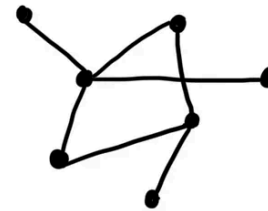
Data in Economics

Data take different structures depending on their purpose.

RECTANGULAR

id	name	number	has_pet
1	Amy	123-4567	1
2	Bob	246-8101	0
3	Carl	135-7911	0
4	Dave	765-4321	1

NON-RECTANGULAR



```
{ "users": [
  {
    "name": "Jasmin",
    "membersince": "2024"
  },
  ...
]
```



Rectangular data

- **Rectangular data** refers to a data structure where information is organized into **rows and columns**.
 - Each row represents an observation or instance of the data.
 - Each column represents a variable or feature of the data.

Rectangular data

- **Rectangular data** refers to a data structure where information is organized into **rows and columns**.
 - CSV (typical for rectangular/table-like data) and variants of CSV (tab-delimited, fix length etc.)
 - Excel spreadsheets (**.xls**)
 - Formats specific to statistical software (SPSS: **.sav**, STATA: **.dat**, etc.)
 - Built-in R datasets
 - Binary formats

Non-rectangular data

- **Hierarchical data** (xml, html, json, etc.)
 - XML and JSON (useful for complex/high-dimensional data sets)
 - HTML (a markup language to define the structure and layout of webpages)
- Unstructured **text** data
- **Internet-of-Things** data
- **Images/Pictures** data

Rectangular data formats

Table-like formats

Example: `ch_gdp.csv`.

```
year,gdp_chfb  
1980,184  
1985,244  
1990,331  
1995,374  
2000,422  
2005,464
```

What is the structure?

Table-like formats

- Reoccurring pattern:
 - Special character ,
 - New lines
 - Table is visible from structure in raw text file...

How can we instruct a computer to read this text as a 6-by-2 table?

Using a simple parser algorithm

- **Parser**: sequence of instructions/rules of how to distinguish columns and rows when all that a computer can do is sequentially read one character after the other.
- Possible if we can agree on certain **standards** of how data structured as a table/matrix is stored in a text file.

Using a simple parser algorithm

In this case: 'Comma-Separated Values' (.csv)

- Commas separate values
- Other delimiters (;, tabs, etc.) possible
- New lines separate rows/observations

Instructions of how to read a .csv-file: CSV parser.

Using a simple CSV parser

Takes a text file with CSV standard structure as input and builds a data structure in the form of a table (represented in RAM).

Look for...

- Commas delimit values/fields in a row
- The end of a line indicates the end of a row

```
00000000: efbb bf79 6561 722c 6764 705f 6368 6662 ...year,gdp_chfb
00000010: 0d31 3938 302c 3138 340d 3139 3835 2c32 .1980,184.1985,2
00000020: 3434 0d31 3939 302c 3333 310d 3139 3935 44.1990,331.1995
00000030: 2c33 3734 0d32 3030 302c 3432 320d 3230 ,374.2000,422.20
00000040: 3035 2c34 3634 05,464
```

CSVs and fixed-width format

- Common format to store and transfer data.
- Very common in a data analysis context.
- Natural format/structure when the dataset can be thought of as a table.

How can **R** read this structure?

Importing Rectangular Data from Text-Files

Comma Separated Values (CSV)

Consider the **swiss**-dataset stored in a CSV:

```
"District","Fertility","Agriculture","Examination","Education","Catholic","Infant.Mortality"  
"Courtelary",80.2,17,15,12,9.96,22.2
```

What do we need to read this format properly?

Parsing CSVs in R

- `read.csv()` (basic R distribution)
- Returns a `data.frame`

Parsing CSVs in R

- Alternative 1: `read_csv()` (`readr`/`tidyr`-package)
 - Returns a `tibble`.
- Alternative 2: The `data.table`-package and the `fread()` function (handling large datasets).

Import and parsing with **readr**

- Why **readr**?
 - Functions for all common rectangular data formats.
 - Consistent syntax.
 - More robust and faster than similar functions in basic R.

Basic usage of **readr** functions

Parse the first lines of the swiss dataset directly like this...

```
library(readr)

read_csv('"District","Fertility","Agriculture","Examination","Education","Catholic","Infant.Mortality"
"Courtelary",80.2,17,15,12,9.96,22.2')
```

```
# A tibble: 1 × 7
  District    Fertility Agriculture Examination Education Catholic Infant.Mortality
  <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1 Courtelary    80.2         17         15         12         9.96        22.2
```

or read the entire **swiss** dataset by pointing to the file

```
swiss <- read_csv("data/swiss.csv")
```

Basic usage of **readr** functions

In either case, the result is a **data frame** (or **tibble**):

```
# A tibble: 47 × 7
  District      Fertility Agriculture Examination Education Catholic Infant.Mortality
  <chr>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
1 Courtelary    80.2           17            15            12            9.96          22.2
2 Delemont      83.1           45.1           6             9            84.8          22.2
3 Franches-Mnt  92.5           39.7           5             5            93.4          20.2
4 Moutier       85.8           36.5           12            7            33.8          20.3
5 Neuveville    76.9           43.5           17            15            5.16          20.6
6 Porrentruy    76.1           35.3           9             7            90.6          26.6
7 Broye         83.8           70.2           16            7            92.8          23.6
8 Glane         92.4           67.8           14            8            97.2          24.9
9 Gruyere       82.4           53.3           12            7            97.7          21
10 Sarine       82.9           45.2           16            13           91.4          24.4
# i 37 more rows
```

Basic usage of **readr** functions

- Other **readr** functions have practically the same syntax and behavior.
- **read_tsv()** (tab-separated)
- **read_fwf()** (fixed-width)
- ...

Data structures and data types in R

Parsing CSVs

Recognizing columns and rows is one thing...

```
# A tibble: 47 × 7
  District      Fertility Agriculture Examination Education Catholic Infant.Mortality
  <chr>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
1 Courtelary    80.2           17            15            12            9.96          22.2
2 Delemont     83.1           45.1           6             9            84.8          22.2
3 Franches-Mnt 92.5           39.7           5             5            93.4          20.2
4 Moutier      85.8           36.5           12            7            33.8          20.3
5 Neuveville   76.9           43.5           17            15            5.16          20.6
6 Porrentruy   76.1           35.3           9             7            90.6          26.6
7 Broye        83.8           70.2           16            7            92.8          23.6
8 Glane        92.4           67.8           14            8            97.2          24.9
9 Gruyere      82.4           53.3           12            7            97.7          21
10 Sarine      82.9           45.2           16            13           91.4          24.4
# i 37 more rows
```

What else did `read_csv()` recognize?

Data types in R: Concept refresher

- Recall the introduction to data types and atomic vectors in Lecture 2 
- R represents data in RAM as **vectors**, which have a
 - Type: `character`, `numeric`, etc.
 - Structure: `data.frame`/`tibble`, etc.
- Parsers in `read_csv()` guess the data types.

Parsing CSV-columns

- "12:00": type character?

Parsing CSV-columns

- "12:00": type `character`?
- What about `c("12:00", "midnight", "noon")`?

Parsing CSV-columns

- "12:00": type `character`?
- What about `c("12:00", "midnight", "noon")`?
- And now `c("12:00", "14:30", "20:01")`?

Parsing CSV-columns

... Let's test it!

```
read_csv('A,B
          12:00, 12:00
          14:30, midnight
          20:01, noon')
```

```
# A tibble: 3 × 2
  A      B
  <time> <chr>
1 12:00  12:00
2 14:30  midnight
3 20:01  noon
```

How can `read_csv()` distinguish the two cases?

Parsing CSV-columns: guess types

- `read_csv()` guesses the column type by inspecting the first 1000 rows and matching patterns: numeric, date/time, logical, character, etc.
- Under the hood `read_csv()` used the `guess_parser()`- function to determine which type the two vectors likely contain:

```
guess_parser(c("12:00", "midnight", "noon"))
```

```
[1] "character"
```

```
guess_parser(c("12:00", "14:30", "20:01"))
```

```
[1] "time"
```

```
parse_time(c("12:00", "14:30", "20:01"))
```

```
12:00:00  
14:30:00  
20:01:00
```

Parsing CSV-columns: guess types

- `read_csv()` guesses the column type by inspecting the first 1000 rows and matching patterns: numeric, date/time, logical, character, etc.
- Under the hood `read_csv()` used the `guess_parser()`- function to determine which type the two vectors likely contain:

```
guess_parser("1'300'000")
```

```
[1] "character"
```

```
guess_parser("1'300'000", locale = locale(grouping_mark = "'"))
```

```
[1] "number"
```

R data structures in depth

Structures to work with...

We now know how data enters R. Let's look at how R stores it internally.

- Data structures for storage on hard drive (e.g., csv).
- Representation of data in RAM (e.g. as an R-object)
 - What is the representation of the 'structure' once the data is parsed (read into RAM)?

Structures to work with...

In the second lecture, we looked at **atomic** vectors as the basic building blocks in **R**.

Structures to work with (in R)

In the second lecture, we looked at **atomic** vectors as the basic building blocks in **R**.

We distinguish two basic characteristics:

1. Data types:

- **integers**;
- **real numbers** ('numeric values', 'doubles', floating point numbers);
- **characters** ('string', 'character values');
- **booleans**

Structures to work with (in R)

We distinguish two basic characteristics:

1. Data types.
2. Basic data structures in RAM:
 - Vectors
 - Arrays/Matrices
 - Lists
 - Data frames (very R-specific)

Structures to work with in R

- A data structure in R is a way of **organizing data in memory**.
- It defines how elements are grouped, stored, and accessed.
- R has a few core structures, each designed for a different kind of organization.

Structure	Key idea	Type	Dim.
Vector	Ordered collection of values	Homog.	1D
Matrix	2D rectangular collection	Homog.	2D
Array	Generalization of a matrix	Homog.	nD
List	Elements can differ in type/size	Heterog.	Flex
Data frame/Tibble	Equal-length column vectors (table)	Heterog.	2D

Describe data

The **type** and the **class** of an object can be used to describe an object.

- **type**: technical and low-level description of the actual storage mode or physical representation of an object.
 - It tells **how the object is stored in memory**.
- **class**: attribute about the nature of an **R** object.
 - It tells you **how to treat the object in a broad sense**.

Data types: numeric and integers

R interprets these bytes of data as type `double` ('numeric') or type `integer`:

```
a <- 1.5  
b <- 3  
c <- 3L  
  
# Use math operators  
a + b
```

```
[1] 4.5
```

Data types: numeric and integers

```
a <- 1.5  
typeof(a); class(a)
```

```
[1] "double"
```

```
[1] "numeric"
```

```
c <- 3L  
typeof(c); class(c)
```

```
[1] "integer"
```

```
[1] "integer"
```

Data types: character

```
a <- "1.5"  
b <- "3"
```

```
typeof(a)
```

```
[1] "character"
```

```
class(a)
```

```
[1] "character"
```

Now the same line of code as above will result in an error:

```
a + b
```

```
Error in a + b: non-numeric argument to binary operator
```

Data types: special values

- **NA**: “Not available”, i.e. missing value for any type
- **NaN**: “Not a number”: special case of NA for numeric
- **Inf**: specific to numeric
- **NULL**: absence of value

Data structures: vectors

- Collections of value of same type

```
persons <- c("Andy", "Brian", "Andy")  
persons
```

```
[1] "Andy" "Brian" "Andy"
```

```
ages <- c(24, 50, 30)  
ages
```

```
[1] 24 50 30
```


Data structures: vectors

What happens when you create a vector out of `persons` and `ages`?

```
c(persons, ages)
```

“Coercion”: Process of converting an object from one data type to another.

- *Implicit*: happens automatically when combining objects of different types R will implicitly coerce to the most flexible type.
- *Explicit*: use specific functions to manually convert data types `as.numeric()`, `as.character()`, `as.logical()`, etc.

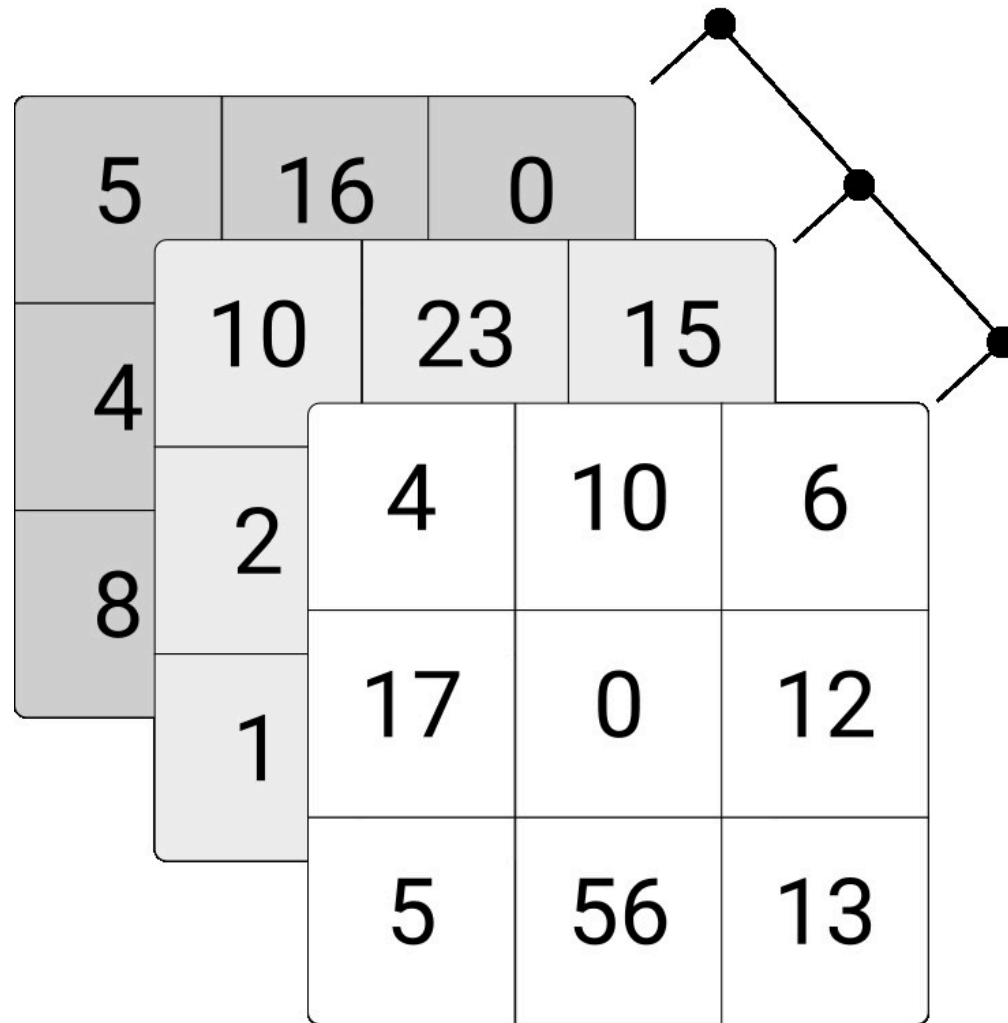
Data structures: matrices

- Matrices are two-dimensional collections of values of the same type
- Like vectors, they are homogeneous
- They are vectors that have a dimension attribute `dim`

1	4	7
2	5	8
3	6	9

Data structures: arrays

Arrays are higher-dimensional collections of values of the same type



Data structures: matrices/arrays

Example:

```
my_matrix <- matrix(c(1,2,3,4,5,6), nrow = 3)
my_matrix
```

```
      [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

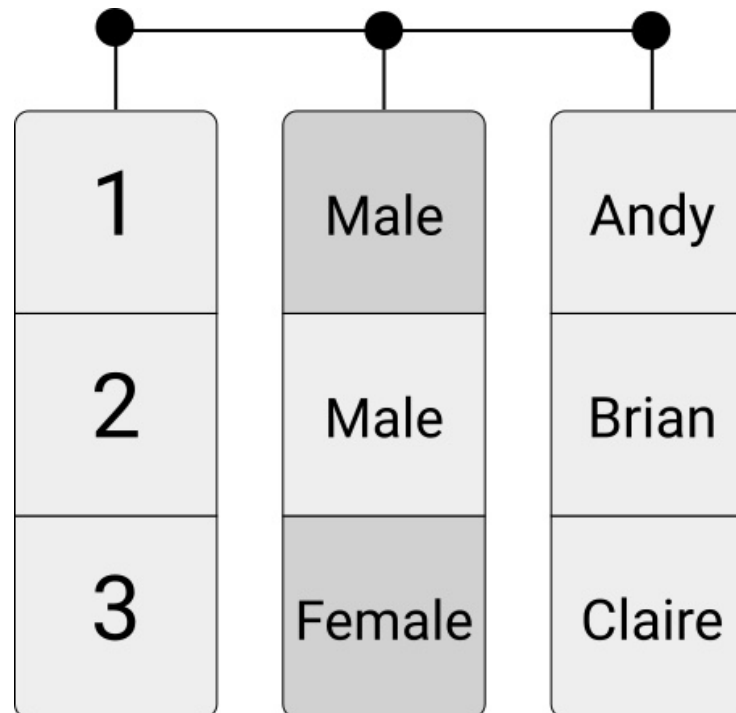
Data structures: matrices/arrays

```
my_array <- array(c(1,2,3,4,5,6,7,8), dim = c(2,3,4))  
my_array
```

```
, , 1  
  
  [,1] [,2] [,3]  
[1,]   1   3   5  
[2,]   2   4   6  
  
, , 2  
  
  [,1] [,2] [,3]  
[1,]   7   1   3  
[2,]   8   2   4  
  
, , 3  
  
  [,1] [,2] [,3]  
[1,]   5   7   1  
[2,]   6   8   2  
  
, , 4  
  
  [,1] [,2] [,3]  
[1,]   3   5   7  
[2,]   4   6   8
```

Data frames, tibbles, and data tables

- Each column contains a vector of a given data type (or factor), but all columns need to be of identical length.
- Under the hood, a data frame is a list of equal-length vectors.
- `data.frame`, `tibble`, `data.table`



Data frames, tibbles, and data tables

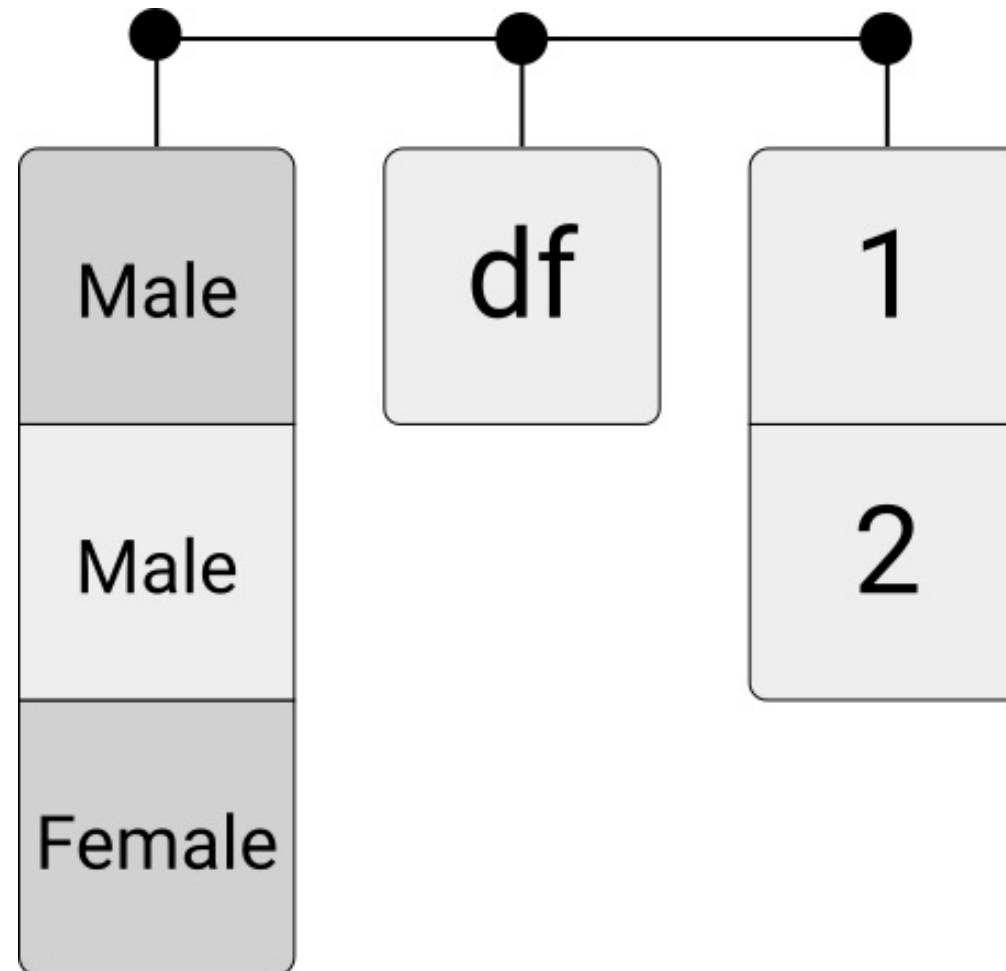
Example:

```
df <- data.frame(person = persons, age = ages, gender = c(0,1,1))  
df
```

	person	age	gender
1	Andy	24	0
2	Brian	50	1
3	Andy	30	1

Data structures: lists

Lists can contain different data types in each element, or even different data structures of different dimensions.



Data structures: lists

Example:

```
my_list <- list(my_array, my_matrix, df)
my_list
```

```
[[1]]
, , 1

      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

, , 2

      [,1] [,2] [,3]
[1,]    7    1    3
[2,]    8    2    4

, , 3

      [,1] [,2] [,3]
[1,]    5    7    1
[2,]    6    8    2

, , 4

      [,1] [,2] [,3]
[1,]    3    5    7
```

Data attributes

Most common attributes:

- names and dimnames
- dim
- class
- levels
- length

Data attributes

```
x <- c(a=1, b=2, c=3)
names(x)
```

```
[1] "a" "b" "c"
```

```
m <- matrix(1:6, nrow=2)
dim(m)
```

```
[1] 2 3
```

```
dimnames(m) <- list(c("row1", "row2"), c("col1", "col2", "col3"))
m
```

```
      col1 col2 col3
row1     1     3     5
row2     2     4     6
```

Data structures: factors

- Factors are sets of categories.
- The values come from a fixed set of possible values.



Data structures: factors

Example:

```
gender <- factor(c("Male", "Male", "Female"))  
gender
```

```
[1] Male   Male   Female  
Levels: Female Male
```

Factors are “disguised” integers...

```
typeof(gender)
```

```
[1] "integer"
```

Data structures: factors

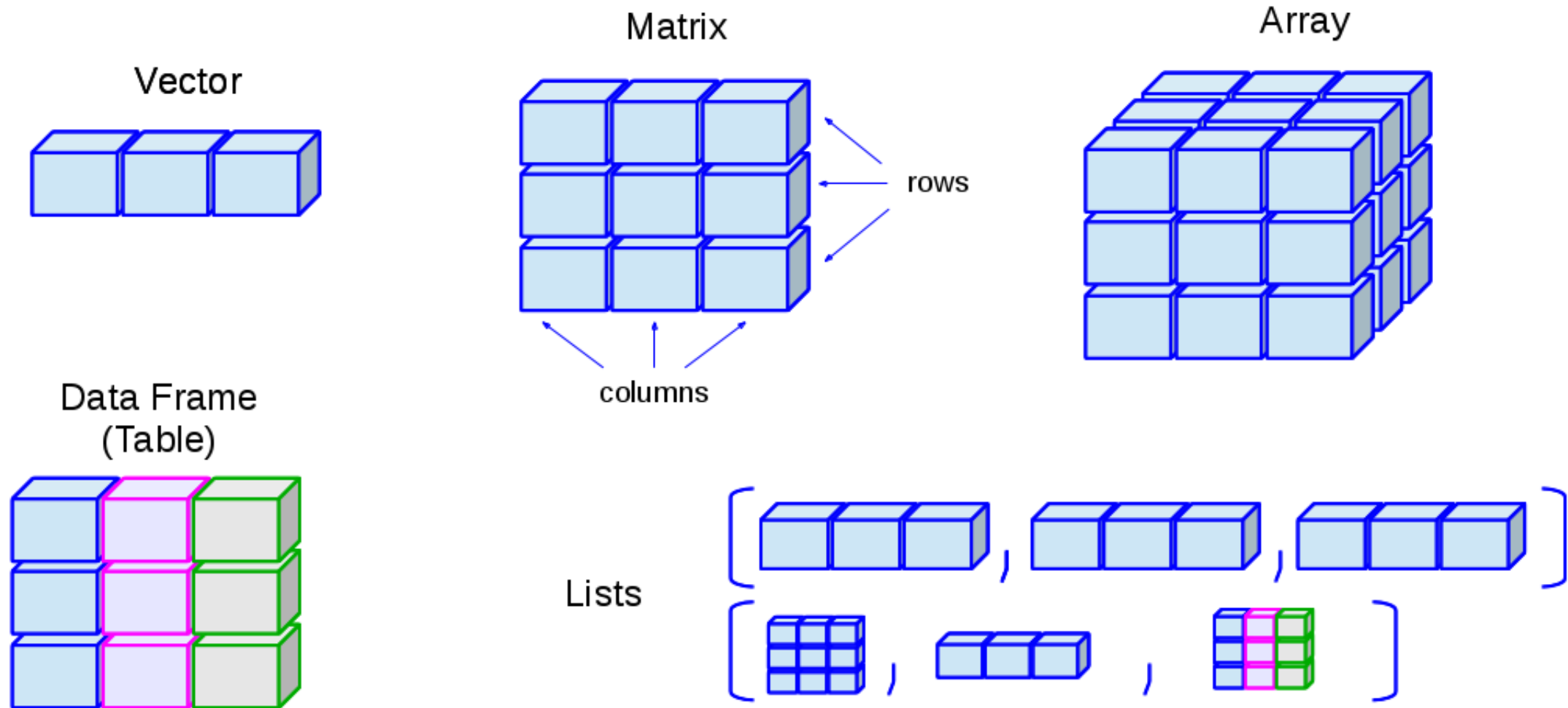
Two components:

- the integer (or “levels”);
- the labels.

```
levels(gender)
```

```
[1] "Female" "Male"
```

Different data types in one figure



Source: <http://venus.ifca.unican.es/Rintro/dataStruct.html>

Manipulating rectangular data in R

Loading built-in datasets

Re-load the `swiss` dataset, or load the built-in dataset.

In order to load built-in datasets, simply use the `data()`-function.

```
data(swiss)
swiss <- read_csv("../..data/swiss.csv")
```

Famous built-in datasets are `mtcars`, `iris`, `USArrests`, etc. The `swiss` dataset is loaded from the package `datasets`, which can be installed with the command `install.packages("datasets")`. Description of the dataset is [here](#).

Manipulations with data frames

```
# inspect the structure  
str(swiss)
```

```
'data.frame':   47 obs. of  7 variables:  
 $ District      : chr  "Courtelary" "Delemont" "Franches-Mnt" "Moutier" ...  
 $ Fertility      : num  80.2 83.1 92.5 85.8 76.9 76.1 83.8 92.4 82.4 82.9 ...  
 $ Agriculture    : num  17 45.1 39.7 36.5 43.5 35.3 70.2 67.8 53.3 45.2 ...  
 $ Examination    : num  15 6 5 12 17 9 16 14 12 16 ...  
 $ Education      : num  12 9 5 7 15 7 7 8 7 13 ...  
 $ Catholic       : num  9.96 84.84 93.4 33.77 5.16 ...  
 $ Infant.Mortality: num  22.2 22.2 20.2 20.3 20.6 26.6 23.6 24.9 21 24.4 ...
```

```
# look at the first few rows  
head(swiss, n = 5)
```

	District	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
1	Courtelary	80.2	17.0	15	12	9.96	22.2
2	Delemont	83.1	45.1	6	9	84.84	22.2
3	Franches-Mnt	92.5	39.7	5	5	93.40	20.2
4	Moutier	85.8	36.5	12	7	33.77	20.3
5	Neuveville	76.9	43.5	17	15	5.16	20.6

Work with data.frames

Select columns

```
swiss$Fertility # use the $-operator
```

```
swiss[, 1] # use brackets [] and the column number/index
```

```
swiss[, "Fertility"] # use the name of the column
```

```
swiss[, c("Fertility", "Agriculture")] # use the name of the column
```

Select rows

```
swiss[1,] # First row
```

```
swiss[swiss$Fertility > 40,] # Based on condition ("filter")
```

Other Common Rectangular Formats

Spreadsheets/Excel

Needs the additional R-package: `readxl`. Then we use the package's `read_excel()`-function to import data from an excel-sheet.



```
# install the package
install.packages("readxl")

# load the package
library(readxl)

# import data from a spreadsheet
swiss_imported <- read_excel("data/swiss.xlsx")
```

Write an Excel Spreadsheet

- Use `openxlsx` to write, style, and edit .xlsx files.
- Alternative to the xlsx package with no dependency on Java.



Data from other data analysis software

- STATA, SPSS, etc.
- Additional packages needed:
 - `foreign`
 - `haven`
- Parsers (functions) for many foreign formats.
 - For example, `read_spss()` for SPSS' `.sav`-format.

```
# Load library
library(haven)

# Read file from SPSS
swiss_imported <- read_spss("data/swiss.sav")
```

Exercise

Exercise

- Download the `financial_data.txt` from Canvas

Appendix: a side note on data frames and tibbles

Rectangular data in R and tidyverse

- data frames: base R
- tibbles: tidyverse

Tibbles are a modern version of data frames

Similar!

- Tibbles are used in the tidyverse and ggplot2 packages.
- Same information as a data frame.
- Small differences in the manipulation and representation of data.
- See [Tibble vs. DataFrame](#) for more details.

Tibbles are a modern version of data frames

```
# A tibble: 47 × 7
  District      Fertility Agriculture Examination Education Catholic Infant.Mortality
  <chr>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>         <dbl>
1 Courtelary    80.2           17            15            12            9.96          22.2
2 Delemont      83.1           45.1           6             9            84.8          22.2
3 Franches-Mnt  92.5           39.7           5             5            93.4          20.2
4 Moutier       85.8           36.5           12            7            33.8          20.3
5 Neuveville    76.9           43.5           17            15            5.16          20.6
6 Porrentruy    76.1           35.3           9             7            90.6          26.6
7 Broye         83.8           70.2           16            7            92.8          23.6
8 Glane         92.4           67.8           14            8            97.2          24.9
9 Gruyere       82.4           53.3           12            7            97.7          21
10 Sarine       82.9           45.2           16            13           91.4          24.4
# i 37 more rows
```

Tibbles are a modern version of data frames

	District	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
1	Courtelary	80.2	17.0	15	12	9.96	22.2
2	Delemont	83.1	45.1	6	9	84.84	22.2
3	Franches-Mnt	92.5	39.7	5	5	93.40	20.2
4	Moutier	85.8	36.5	12	7	33.77	20.3
5	Neuveville	76.9	43.5	17	15	5.16	20.6
6	Porrentruy	76.1	35.3	9	7	90.57	26.6
7	Broye	83.8	70.2	16	7	92.85	23.6
8	Glane	92.4	67.8	14	8	97.16	24.9
9	Gruyere	82.4	53.3	12	7	97.67	21.0
10	Sarine	82.9	45.2	16	13	91.38	24.4
11	Veveyse	87.1	64.5	14	6	98.61	24.5
12	Aigle	64.1	62.0	21	12	8.52	16.5
13	Aubonne	66.9	67.5	14	7	2.27	19.1
14	Avenches	68.9	60.7	19	12	4.43	22.7
15	Cossonay	61.7	69.3	22	5	2.82	18.7
16	Echallens	68.3	72.6	18	2	24.20	21.2
17	Grandson	71.7	34.0	17	8	3.30	20.0
18	Lausanne	55.7	19.4	26	28	12.11	20.2
19	La Vallee	54.3	15.2	31	20	2.15	10.8
20	Lavaux	65.1	73.0	19	9	2.84	20.0
21	Morges	65.5	59.8	22	10	5.23	18.0
22	Moudon	65.0	55.1	14	3	4.52	22.4

Tibbles vs data frames

- Unlike data frames, tibbles don't show the entire dataset when you print it
- Tibbles cannot access a column when you provide a partial name of the column, but data frames can.
- When you access only one column of a tibble, it will keep the tibble structure. But when you access one column of a data frame, it will become a vector.
- When assigning a new column to a tibble, the input will not be recycled, which means you have to provide an input of the same length of the other columns. But a data frame will recycle the input.
- Tibbles preserve all the variable types, while data frames have the option to convert string into factor. (In older versions of R, data frames will convert string into factor by default)